Data Normalization & LLM Extraction

Solution Architecture, Methodology and Run Instructions

Python Developer Intern Assignment | Author: Ramesh

1. Overview

The task is to turn a raw, scraped JSON payload of telecom products (device, variant, pricing and unstructured promotional text) into a strictly formatted CSV with **29 exact columns**. Every product is evaluated twice, once as a **New Line** and once as an **Upgrade**, so N products always produce exactly **2N rows**.

The solution is a **hybrid pipeline**: deterministic Python rules handle every field that has a precise, checkable definition, and an LLM (via OpenRouter) is used only for the fuzzy, free-text fields. A validation layer then removes anything the model invented, duplicated or got wrong before the row is written.

Key goals: correct line-type eligibility, strict variant matching, no hallucination, exact decimals, and robust handling of malformed LLM output.

2. Project Structure

```
ASSIGNMENT/
|-- data/
|   |-- output/
|   |   `-- normalized_output.csv   <- generated result
|   `-- Input.json                  <- raw scraped product data
|-- .env                            <- API key and model (not committed)
|-- main.py                         <- the entire pipeline
`-- requirements.txt                <- requests, python-dotenv
```

3. How to Run (Step by Step)

Step 1 - Create the .env file

In the project root create a file named `.env` and add your OpenRouter API key (the model line is optional; the default is shown):

```
OPENROUTER_API_KEY=your_api_key_here
OPENROUTER_MODEL=meta-llama/llama-3.3-70b-instruct
```

Step 2 - Install dependencies

From the project root run:

```
pip install -r requirements.txt
```

Step 3 - Test the rule-based logic first (no API needed)

This runs only the deterministic rules; no network calls are made:

```
python main.py --no-llm
```

Step 4 - Full run with the LLM

Runs rules plus LLM extraction and validation:

```
python main.py
```

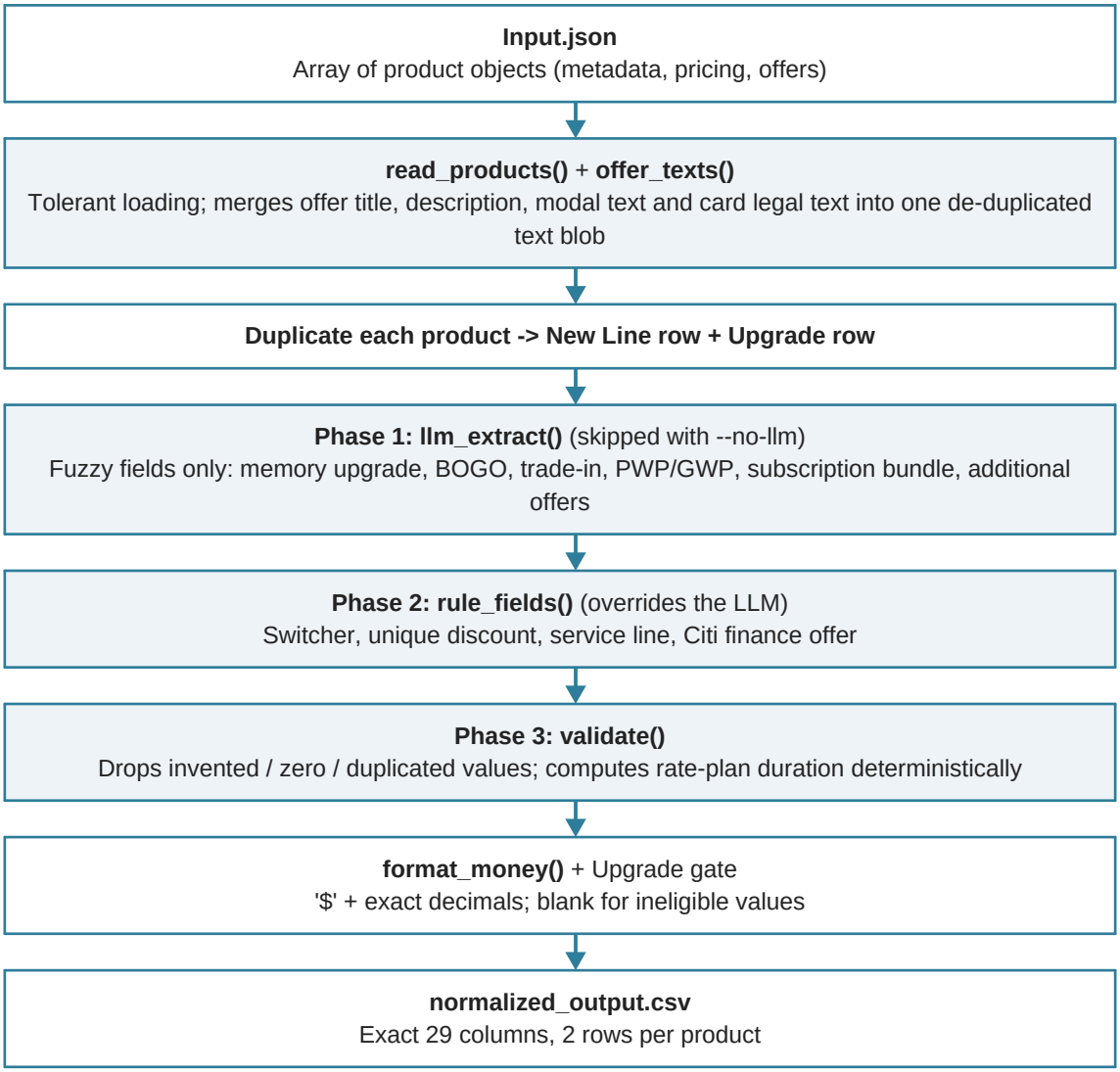
Optional arguments

```
python main.py --input data/Input.json --output data/output/normalized_output.csv
```

The result is written to `data/output/normalized_output.csv` (UTF-8, no BOM, so headers stay exact). On success the script prints `Done: N rows, 29 columns` and asserts that rows = products x 2.

4. Solution Architecture

The pipeline runs per product and per line type. Rules and the LLM both write into a single row dictionary; the rules are applied *after* the LLM so they always override it.



4.1 Module Breakdown

Function	Responsibility
get()	Safe nested-key lookup with fallbacks (e.g. product_name, then name). Never raises on missing keys.
offer_texts()	Collects each offer's title, description, modal_raw_text and card text; de-duplicates; ignores accessory arrays.
variant_pattern()	Builds a regex from the variant (e.g. 256 GB becomes 256\s*GB) so figures are anchored to the exact storage size.
rule_fields()	All deterministic extraction: switcher, unique discount, service line, Citi finance offer.
llm_payload()	Sends the model only product name, variant, pricing and trimmed offer texts (max 6000 chars each); filters out statement-credit and break-your-contract blocks already handled by rules.
llm_extract()	Calls OpenRouter with temperature 0, a strict system prompt, 3 retries with back-off; returns blanks on total failure.
parse_json()	Tolerant parser: handles None, markdown code fences and text around the JSON object.
validate()	Post-LLM guardrails (see section 5.4) and rate-plan duration.
format_money()	Converts to '$' + exact decimals using Decimal; no rounding, no thousands separators.
make_row()	Orchestrates one row: base fields, LLM, rules, validation, Upgrade gate, formatting.
main()	CLI, product loop (New Line then Upgrade), CSV writing, row-count assertion.

5. Methodology

5.1 Why a hybrid approach?

Fields such as the Citi finance value or a variant-specific bill credit depend on exact numbers and strict eligibility rules. LLMs can mis-add, mix up variants or invent values, so these are computed with regex and Decimal arithmetic. Fields that need reading comprehension (what counts as a gift-with-purchase, a subscription bundle or a BOGO) are delegated to the LLM, then verified in Python.

Handled by Python rules	Handled by the LLM (then validated)
SWITCHER, SWITCHER_OFFER_TEXT UNIQUE DISCOUNT SERVICE LINE OFFERS and $ VALUE FINANCE OFFER DESCRIPTION 1 and $ VALUE RATEPLAN DURATION (MONTHS) All base fields (name, brand, variant, prices, shipment, stock)	FREE MEMORY UPGRADE, BOGO OFFER Max TI and Trade-In Value Type PWP/GWP OFFERS, Cleaned PWP/GWP OFFERS, $ VALUE SUBSCRIPTION BUNDLE OFFERS and $ VALUE ADDITIONAL OFFERS and $ VALUE

5.2 Line-type logic (New Line vs Upgrade)

- **Switcher:** port-in based, so it is evaluated for New Line only and always blank for Upgrade.
- **Global eligibility gate:** if the text contains *"Eligibility: New customers activating new line"*, the device bill-credit block (unique discount) is valid only for New Line; Upgrade is blocked.
- **Service line credit:** New Line only. Matches "additional $N in credits", otherwise "Get $N off a new line".
- **Finance (Citi):** the welcome bonus applies to New Line only; the yearly statement credit and the monthly bill discount apply to both.

- **LLM prompt:** told which line type it is evaluating and that new-line / port-in / new-customer offers are not eligible for Upgrade.

5.3 Strict variant matching and hallucination prevention

- Unique discount is extracted with a regex anchored to the product's own variant followed by the *"for $X/mo. for N mos. after up to $Y in bill credits"* pattern, so a 256GB row can never pick up the 512GB or 1TB credit.
- The LLM prompt states: use only this exact variant, never guess, never use outside knowledge, never sum or compute amounts, keep exact decimals, output JSON only.
- Temperature is 0 for deterministic output.
- Every LLM-produced dollar amount must appear literally in the source text; otherwise it is cleared.
- Max TI is populated only when an explicit maximum dollar amount is stated (generic trade-in instructions give blank).

5.4 Validation layer (validate)

#	Guardrail	Effect
1	LLM money fields (Max TI, PWP/GWP, Subscription, Additional) must be non-zero and appear in the source text	Invented or zero values are blanked
2	Additional offer must not duplicate another category's value (service line, unique discount, switcher, finance) or be a fee/headline (next up anytime, bill credit, off a new line, for $0)	Enforces the non-duplication rule
3	Rate-plan duration taken from "N-month installment", else "for N mos", else "after N monthly bill credits"	Deterministic, same rule for both line types
4	Upgrade rows have SWITCHER and SWITCHER_OFFER_TEXT forced blank	Safety net for new-line-only offers

5.5 Formatting and null handling

- All money columns go through **format_money()**: leading '$', exact decimals as in the source (for example $800, $244.36), computed with Decimal so nothing is rounded or truncated.
- Missing or disqualified values are left as blank cells. No "N/A", "0" or "-" placeholders.
- The CSV is written without a BOM and with exactly the 29 required headers in the required order.

5.6 Error handling and robustness

- **JSON input:** accepts a plain array or an object containing products / items / data; raises a clear error otherwise; non-dict items are skipped.
- **LLM output:** parse_json handles empty content, code fences and surrounding text; malformed output triggers a retry.
- **Network / API:** timeout of 120 s; up to 3 attempts with increasing back-off; API error bodies are detected; an unavailable model (HTTP 404) stops immediately with a clear message.
- **Graceful degradation:** if all retries fail, only the LLM columns for that row are left blank and the rule-based columns are still produced.
- **Startup checks:** missing API key (suggests --no-llm) and missing input file.
- **Integrity check:** assertion that output rows equal products x 2.

6. Worked Example: Apple iPhone 17 Pro Max (256 GB)

The expected outputs from the assignment are reproduced by the rules as follows.

Column	New Line	Upgrade
PRODUCT NAME / BRAND / VARIANT / MSRP	Apple iPhone 17 Pro Max / Apple / 256 GB / from pricing.retail	Same
UNIQUE DISCOUNT	$660.36 (256GB match, new-line eligibility passes)	blank (global gate blocks)
SERVICE LINE OFFERS / $ VALUE	Price after additional $200 in credits. / $200	blank
Max TI	blank (no explicit maximum dollar amount)	blank
SWITCHER	$800	blank
SWITCHER_OFFER_TEXT	Get up to $800/line when you switch to AT&T and get a reward card to pay off your phone.	blank
FINANCE OFFER $ VALUE	$450 = $200 welcome + $240 yearly + $10 discount	$250 = $240 + $10
FINANCE OFFER DESCRIPTION 1	Earn a $200 statement credit after qualifying purchases welcome bonus plus up to $240 back every year with the AT&T Points Plus® Card from Citi and $10 discounts on your AT&T bills	Get up to $240 back every year with the AT&T Points Plus® Card from Citi and $10 discounts on your AT&T bills

7. Output Schema (29 Columns)

Group	Columns	Source
Base	PLAN TYPE, PRODUCT NAME, BRAND, VARIANT, BASE, MSRP, RESELLER LIST PRICE, SHIPMENT, STOCK	JSON fields (BASE = pricing.monthly, MSRP = pricing.retail)
Line	LINE TYPE, RATEPLAN DURATION (MONTHS)	Loop / validate()
Carrier offers	SWITCHER, SWITCHER_OFFER_TEXT, UNIQUE DISCOUNT, SERVICE LINE OFFERS, SERVICE LINE OFFER $ VALUE	Rules
Finance	FINANCE OFFER DESCRIPTION 1, FINANCE OFFER $ VALUE	Rules
Fuzzy offers	FREE MEMORY UPGRADE, BOGO OFFER, Max TI, Trade-In Value Type, PWP/GWP (3 columns), SUBSCRIPTION BUNDLE (2), ADDITIONAL (2)	LLM + validation

8. Design Decisions and Limitations

- Rules always override the LLM, so a model mistake can never corrupt a deterministic field.
- Regex patterns are written for the offer wording seen in the provided data; a very different carrier phrasing would need new patterns.
- The model is configurable through OPENROUTER_MODEL, so it can be swapped without touching the code.
- The --no-llm mode makes the rule logic independently testable and the project runnable without any API key.